LinkedList programs :

```
package com.ravi.linked_list;

import java.util.ArrayList;
import java.util.LinkedList;

public class LinkedListDemo1
{
    public static void main(String[] args)
    {
        final int ITERATION = 100000;

        ArrayList<Integer> al = new ArrayList<>();

        long startTime = System.currentTimeMillis();  //2ms

        for(int i=0; i<ITERATION; i++)
        {
          al.add(0, i);
        }

        long endTime = System.currentTimeMillis();  //9ms

        IO.println("Total Time Taken by ArrayList class :"+(endTime - startTime)+" ms");

       LinkedList<Integer> list = new LinkedList<>();

        startTime = System.currentTimeMillis();  //11ms

        for(int i=0; i<ITERATION; i++)
        {
          list.add(0, i);
        }

        endTime = System.currentTimeMillis();

        IO.println("Total Time Taken by LinkedList class :"+(endTime - startTime)+" ms");

    }
}
```

Note : From the above program, It is clear that LinkedList is very good choice for insertion of the objects at the begning of the List

---

```
package com.ravi.linked_list;


import java.util.Arrays;
//Methods of LinkedList class
import java.util.LinkedList;
import java.util.List;
public class LinkedListDemo2
{
  public static void main(String[] argv)
  {

      //Stores the element based on the index in a non contiguous memory location
      LinkedList<Integer> numbers = new LinkedList<>();
      numbers.add(100);
      numbers.add(900);
      numbers.add(1200);
      IO.println(numbers.get(1));



      //Adding OR deleting the element from first & last position
      LinkedList<String> listOfNames = new LinkedList<>();
      listOfNames.add("Scott");
      listOfNames.addFirst("Alen");
      listOfNames.addLast("John");
      IO.println(listOfNames); //[ Scott ]

      listOfNames.removeFirst();
      listOfNames.removeLast();
      IO.println(listOfNames); //[Scott]


       List<String> listOfName = Arrays.asList("Ravi","Rahul","Ankit", "Rahul");
       LinkedList<String> name = new LinkedList<>(listOfName);

       name.sort((s1,s2)-> s1.compareTo(s2));
       name.forEach(IO::println);


  }
}
```

ListIteartor<E> Methods :
----

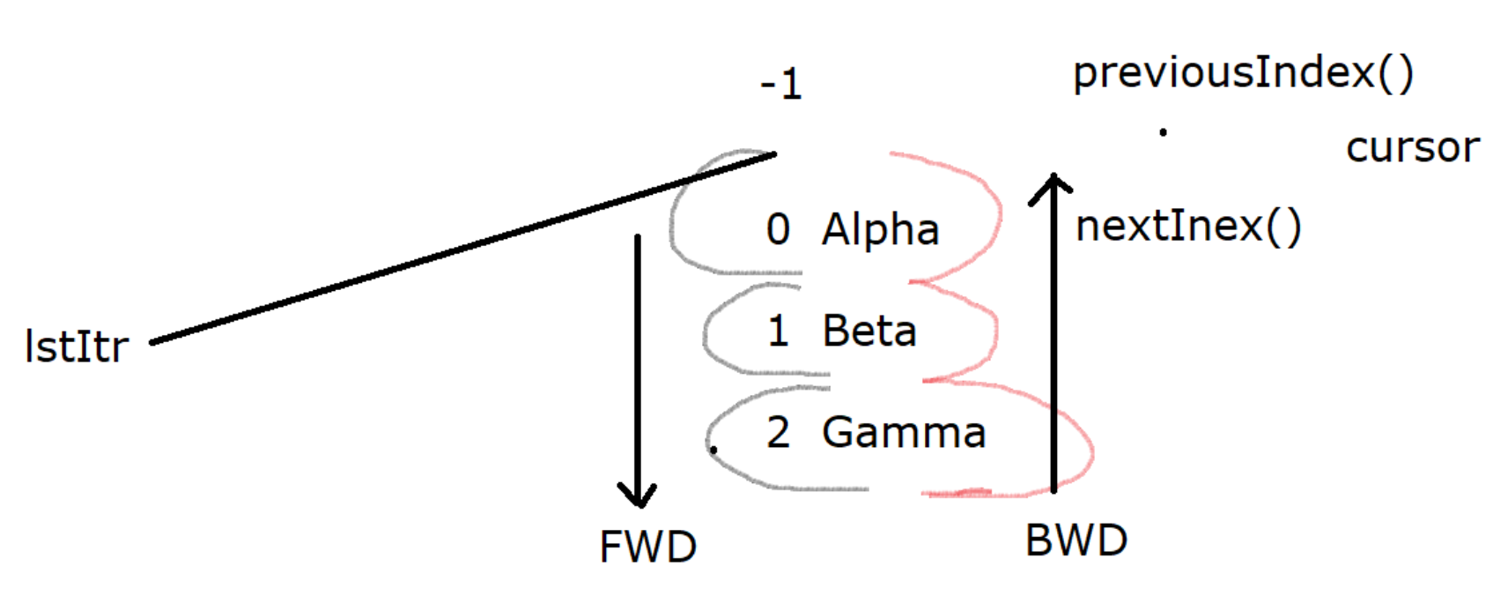
* As we know ListIterator is used to **retrieve the objects** in forward direction as well as backward direction, along with these methods we have few more methods are available
add(E e), remove(), set(E e), nextIndex() and previousIndex()

```
package com.ravi.linked_list;

//ListIterator methods (add(E e), set(E e), remove())
import java.util.*;
public class LinkedListDemo3
{
    public static void main(String[] args)
    {
        LinkedList<String> city = new LinkedList<>();

         city.add("Kolkata");
        city.add("Bangalore");
        city.add("Hyderabad");
        city.add("Pune");

        IO.println(city);

        ListIterator<String> lt = city.listIterator();

         while(lt.hasNext())
          {
            String cityName =  lt.next();

            if(cityName.equals("Kolkata"))
            {
                lt.remove();
            }
            else if(cityName.equals("Hyderabad"))
            {
                lt.add("Ameerpet");
            }
            else if(cityName.equals("Pune"))
            {
                lt.set("Mumbai");
            }
        }

        city.forEach(System.out::println);
    }
}
```

Note : Here there is no ConcurrentModificationException becuase modification is done through own method of ListIterator interface.

```
package com.ravi.linked_list;


//Insertion, deletion, displaying and exit

import java.util.LinkedList;
import java.util.List;
import java.util.Scanner;

public class LinkedListDemo4
{
public static void main(String[] args)
     {
    List<Integer> linkedList = new LinkedList<>();
    Scanner scanner = new Scanner(System.in);

      while (true)
          {
      System.out.println("Linked List: " + linkedList); //[]
      System.out.println("1. Insert Element");
      System.out.println("2. Delete Element");
          System.out.println("3. Display Element");
          System.out.println("4. Clear the list");
      System.out.println("5. Exit");
      System.out.print("Enter your choice: ");

      int choice = Integer.parseInt(scanner.nextLine());

         switch (choice)
              {
         case 1 ->
         {
          IO.print("Enter the element to add :");
          int elementToInsert =  Integer.parseInt(scanner.nextLine());
          linkedList.add(elementToInsert);
          IO.println("Element "+elementToInsert+" inserted successfully");
         }
         case 2 ->

         {
          if(linkedList.isEmpty())
          {
               IO.println("Linkedlist is empty, Nothing to delete ");
          }
          else
          {
               IO.print("Enter the element to delete :");
               int elementToDelete =  Integer.parseInt(scanner.nextLine());
               boolean isRemoved =  linkedList.remove(Integer.valueOf(elementToDelete));
               if(isRemoved)
               {
                    IO.println(elementToDelete+" is removed");
               }
               else
               {
                    IO.println(elementToDelete+" is not available to delete");
               }

          }
         }

         case 3->
         {
          IO.println("Linked list elements :");
          linkedList.forEach(IO::println);
         }

         case 4->
         {
          linkedList.clear();
          IO.println("After deleting all the elements "+linkedList);
         }

         case 5->
         {
          IO.println("Exiting from the application");
          System.exit(0);
         }

         default->
            System.out.println("Invalid choice. Please try again.");
      }
    }
}
}
```

```
package com.ravi.linked_list;

import java.util.LinkedList;

public class LinkedListDemo5
{
    public static void main(String[] args)
    {
        LinkedList<String> train = new LinkedList<>();

        train.add("Coach A");
        train.add("Coach B");
        train.add("Coach C");
        IO.println("Initial Train: " + train);

        train.addFirst("Engine");
        IO.println("After adding Engine at front: " + train);


        train.addLast("Guard Coach");
        IO.println("After adding Guard Coach at end: " + train);


        train.add(2, "Pantry Coach");
        IO.println("After adding Pantry Coach in middle: " + train);

        IO.println("Reached To destination");

        train.removeFirst();
        System.out.println("After removing first coach: " + train);


        train.removeLast();
        System.out.println("After removing last coach: " + train);


        train.remove(1);
        System.out.println("After removing Pantry Coach from middle: " + train);

    }
}
```



```
package com.ravi.linked_list;

import java.util.LinkedList;
import java.util.List;
import java.util.ListIterator;
import java.util.NavigableSet;
import java.util.TreeSet;

public class LinkedListDemo6
{
     public static void main(String[] args)
     {
      List<String> list = new LinkedList<>();
      list.add("Alpha"); //0
      list.add("Beta");  //1
      list.add("Gamma"); //2

     ListIterator<String> lstItr = list.listIterator();

     IO.println("Previous index position :"+lstItr.previousIndex());  //-1
     IO.println("Next index position :"+lstItr.nextIndex()); //0

     while(lstItr.hasNext())
     {
       IO.println("Inside the while loop cursor position :"+lstItr.nextIndex());
       IO.println(lstItr.next());
     }

     }
}
```

*** What is the difference between Iterator & ListIterator interface :

Difference between Iterator<E> and ListIterator<E> :

Iterator<E>	ListIterator<E>
1) Can move in forward direction only.	1) Can move in both forward and backward direction.
2) During Iteration, We can only remove the elements.	2) During Iteration, We can remove, add and replace the elements.
3) Can work with List<E>, Set<E> and Queue<E> interface. (Universal Iterator)	3) Can work with only List<E> interface.
4) We cannot get index position of the element.	4) We can get index position of the element by using nextIndex() and previousIndex()
5) Does not provide index technique.	5) It supports indexing technique.

Set<E> interface :
----

* It is the sub interface of Collection interface available from JDK 1.2V.

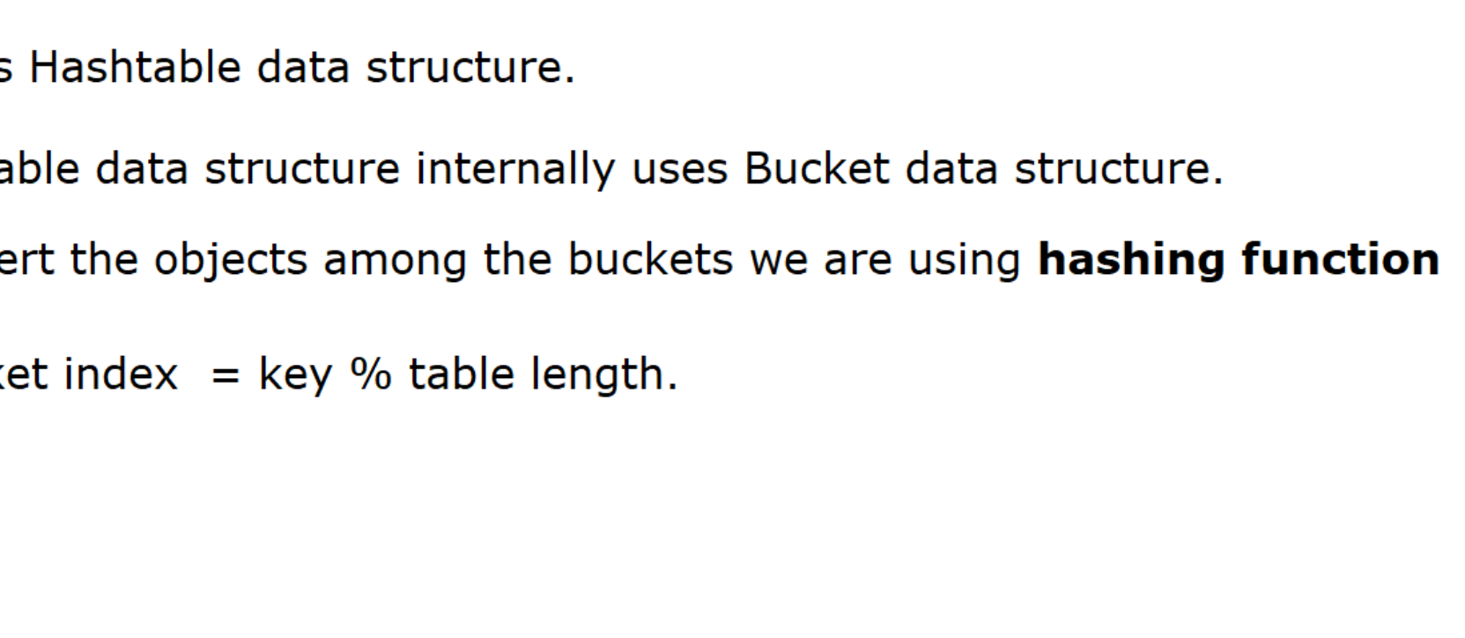
* It does not accept duplicate objects here internally hashCode() and equals(Object obj) contract is working.

* It does not follow **indexing technique** so ListIterator will not work with Set<E> interface.

* It uses hashing algorithm & Red Black tree data straucture.

* It supports all the methods of Collection interface as well as few more methods are added from java 9V.

Set<E> interface Hierarchy :



What is hashing algorithm ?
----

* It is one of the most popular algorithm for **fast searching operation OR fast retrieval operation.**

* The hashing algorithm technique provides O(1) time complexity for all the operations like add, delete & search

* It uses Hashtable data structure.

* Hashtable data structure internally uses Bucket data structure.

* In insert the objects among the buckets we are using **hashing function**

Bucket index = key % table length.